

**MACHINE LEARNING AT THE NETWORK EDGE:
COMPRESSED SENSOR DATA FOR GATEWAY-BASED
ACTIVITY RECOGNITION**

by

CHENHAO XU

A CAPSTONE PROJECT SUBMITTED FOR THE DEGREE OF

MASTER OF COMPUTING

in the

GRADUATE DIVISION

of the

NATIONAL UNIVERSITY OF SINGAPORE

2026

Supervisor:

Prasanna Karthic Vairam

Examiners:

Prasanna Karthic Vairam

Declaration

I hereby declare that this project report is my original work and it
has
been written by me in its entirety. I have duly
acknowledged all the sources of information which have
been used in this report.

This report has also not been submitted for any
degree in any university previously.



Chenhao XU

22 August 2026

Acknowledgments

I would like to thank my supervisor for guidance on the project direction and evaluation criteria. I am grateful to the course staff and peers whose feedback helped turn a broad edge-computing topic into a reproducible experimental study. I also thank my family and friends for their support throughout the capstone period.

Contents

Acknowledgments	i
Abstract	v
List of Figures	vii
List of Tables	viii
1 Introduction	1
1.1 Background and Motivation	1
1.2 Problem Statement	2
1.3 Research Questions	3
1.4 Scope and Assumptions	3
1.5 Contributions	4
1.6 Report Organisation	4
2 Background and Literature Review	5
2.1 From Cloud Computing to Edge Intelligence	5
2.2 Collaborative and Split Inference	6
2.3 Human Activity Recognition at the Edge	7
2.4 A Taxonomy of Compression Families	7
2.4.1 Lossless Byte Compression	8
2.4.2 Numerical Quantisation	8
2.4.3 Principal Component Analysis	9
2.4.4 Autoencoder Representations	9
2.4.5 Task-Aware and Jointly Learned Compression	10
2.5 Model Compression Is a Separate Axis	11

2.6	Synthesis and Research Gap	11
3	Experimental Methodology and Implementation	12
3.1	Experimental Design	12
3.2	Dataset and System Mapping	13
3.3	Baseline Classifier Selection	13
3.4	Transmission Baselines	14
3.4.1	Raw float32	14
3.4.2	Record-wise zlib	14
3.5	Numerical Quantisation	15
3.6	PCA Representation	15
3.7	Autoencoder Representation	16
3.8	Metrics and Decision Rules	16
3.9	Deployment Validation	17
3.10	Reproducibility and Leakage Controls	17
4	Experimental Results	18
4.1	Raw and Lossless Baselines	18
4.2	Numerical Quantisation	18
4.3	PCA Results	20
4.4	Autoencoder Results	21
4.5	Unified Compression-Family Comparison	22
4.6	Application-Oriented Recommendations	24
4.7	Gateway Deployment Proxy	24
5	Discussion	26
5.1	Answers to the Research Questions	26
5.1.1	RQ1: Lossless and Low-Precision Transmission	26
5.1.2	RQ2: Dimensionality and the Quality Frontier	26
5.1.3	RQ3: PCA versus Autoencoder	27
5.1.4	RQ4: Recommended Deployment Points	27
5.2	Systematic Comparison of Compression Families	27
5.3	Interpreting the PCA–AE Crossover	29
5.4	Payload Accounting in a Real Network	29

5.5	Operational Deployment Policy	30
5.6	Privacy, Security and Resilience	30
5.7	Threats to Validity	31
5.7.1	Internal Validity	31
5.7.2	Construct Validity	31
5.7.3	External Validity	31
5.7.4	Conclusion Validity	32
6	Conclusion and Future Work	33
6.1	Conclusion	33
6.2	Future Work	34
	Bibliography	35
A	Reproducibility Details	37
A.1	Repository Structure	37
A.2	Environment and Dependencies	37
A.3	Determinism and Data Integrity	38
A.4	Artifact Checks	38
A.5	Payload Formulae	39

Abstract

Machine Learning at the Network Edge:
Compressed Sensor Data for Gateway-Based Activity Recognition

by

Chenhao XU

Master of Computing in Computer Science

National University of Singapore

Internet of Things (IoT) endpoints increasingly generate data that must be acted upon quickly, yet transferring raw observations to a remote cloud consumes bandwidth and may expose sensitive information. This project investigates whether an IoT gateway can make accurate predictions from compressed sensor data. The UCI Human Activity Recognition Using Smartphones dataset is used as a controlled case study. A smartphone is treated as an IoT endpoint and a local computer as an edge gateway; each record contains 561 engineered inertial-sensor features and one of six activity labels.

The evaluation compares raw float32 transmission, record-wise lossless zlib, float16 and calibrated symmetric int8 quantisation, principal component analysis (PCA), and nonlinear autoencoder (AE) representations. All transforms are fitted only on the official subject-disjoint training partition. The gateway baseline, standardised logistic regression, reaches 95.44% Macro-F1 with a 2,244-byte feature payload. Symmetric int8 reduces the payload to 561 bytes, a $4.00\times$ compression ratio, while retaining 95.50% Macro-F1. At matched latent dimensions PCA is stronger at 128 and 64 components, whereas the AE is stronger at 32, 16 and 8 components. An eight-dimensional AE code uses 32 bytes and retains 86.77% Macro-F1. A TorchScript deployment proxy validates the selected end-to-end int8 path at 95.03% Macro-F1.

The principal recommendation is feature-wise symmetric int8 for deployments that require near-baseline quality. AE representations become defensible only when the link budget is substantially tighter and endpoint compute is available. The findings demonstrate that compressed feature transmission can support useful gateway

inference, while also showing that compression families must be compared using downstream task quality, communication cost, endpoint cost and deployment complexity rather than compression ratio alone.

Keywords: edge intelligence; IoT gateway; sensor compression; quantisation; principal component analysis; autoencoder; human activity recognition.

List of Figures

1.1	System boundary used throughout the report. The measured communication unit is one encoded 561-feature record, not a complete network packet.	2
4.1	Quantisation trade-off. Moving from float32 to float16 and int8 gives deterministic fixed-width payload reductions while the Macro-F1 markers remain effectively level. The figure therefore identifies int8 as a Pareto improvement over the raw message for this dataset and classifier. . . .	19
4.2	PCA quality-compression curve across matched bottleneck dimensions. Each move to the right reduces payload but also removes discriminative information; there is no broad plateau comparable with numerical int8.	21
4.3	Training and validation reconstruction loss for AE bottlenecks. The curves support optimisation stability and checkpoint selection; they do not by themselves establish that the latent codes preserve activity labels.	21
4.4	Autoencoder quality-compression relationship. Quality declines with bottleneck width, but the narrowest codes retain more Macro-F1 than the matched PCA alternatives shown later in Figure 4.5.	22
4.5	Unified payload-Macro-F1 frontier. The upper-left region is preferable. Int8 is the near-lossless operating point; AE-32 and AE-8 extend the frontier when the permitted message is 128 or 32 bytes respectively. . .	24
4.6	Recommended points for three bandwidth-quality policies. The bars do not imply equal endpoint cost: the AE points require a neural encoder, whereas int8 requires only calibrated scaling, rounding and clipping. . .	25
4.7	Gateway deployment comparison. Dynamic int8 reduces classifier storage while retaining high test quality. This model-artifact reduction is reported separately from the $4\times$ feature-message reduction.	25

List of Tables

2.1	Compression families relevant to the endpoint–gateway data path. . . .	8
3.1	Dataset and simulated deployment mapping.	13
4.1	Raw and record-wise lossless baselines.	19
4.2	Numerical-precision results with the raw-space gateway classifier. . . .	19
4.3	PCA payload, explained variance and downstream quality.	20
4.4	Autoencoder latent-space results.	22
4.5	Unified endpoint-to-gateway comparison.	23
4.6	Selections for illustrative application targets.	24
5.1	Qualitative comparison beyond payload and Macro–F1.	28
A.1	Experiment entry points and principal outputs.	37

Chapter 1

Introduction

1.1 Background and Motivation

The Internet of Things (IoT) has shifted a substantial volume of data generation from central data centres to the network periphery. Smartphones, wearables, industrial controllers and environmental sensors continuously observe physical processes near the place where an action may be required. The conventional cloud-centric machine-learning pattern uploads these observations before inference. Although a cloud provides abundant computation and storage, the model can be a poor fit when response time, uplink capacity, monetary cost, intermittent connectivity or exposure of sensitive observations matters. Edge computing moves selected computation closer to the source to address those constraints [1, 2]. Edge intelligence extends the idea to artificial-intelligence workloads and distinguishes intelligence deployed *on* edge infrastructure from intelligence used to manage that infrastructure [3].

This report studies a narrow but practical edge-intelligence problem. An IoT endpoint produces a sensor-feature vector, encodes it into a smaller message and sends the message to a nearby gateway. The gateway decodes the message where necessary and predicts an activity label. The question is not whether remote cloud infrastructure can obtain the highest possible predictive performance. It is whether the bytes crossing the endpoint–gateway link can be reduced without making the subsequent inference unreasonable. This boundary separates *data compression*, which reduces the message, from *model compression*, which reduces the classifier artifact. Both matter in an edge system, but they save different resources.

Human activity recognition (HAR) provides a controlled sensor case study. The UCI HAR dataset was collected from waist-mounted smartphones carried by 30

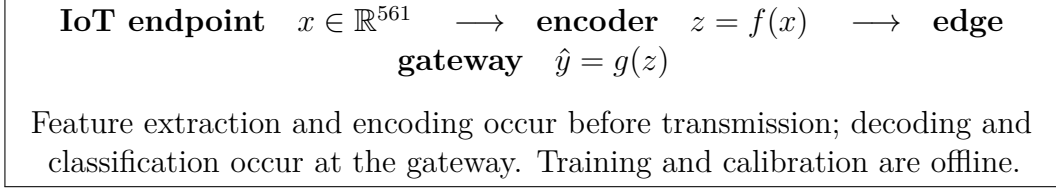


Figure 1.1: System boundary used throughout the report. The measured communication unit is one encoded 561-feature record, not a complete network packet.

participants while they performed six activities of daily living [4, 5]. It provides tri-axial accelerometer and gyroscope measurements transformed into 561 time- and frequency-domain features, together with activity labels and subject identifiers. A smartphone is a plausible IoT endpoint, and activity inference is representative of wearable health, rehabilitation, fitness and ambient-assistance applications. The benchmark is not claimed to reproduce every streaming deployment; it supplies a public, subject-disjoint split with which compression and inference decisions can be compared reproducibly.

1.2 Problem Statement

Let $x \in \mathbb{R}^d$ denote a sensor feature vector, with $d = 561$, and let y be its activity class. At the endpoint, an encoder f produces a transmitted representation $z = f(x)$. At the gateway, a classifier g predicts $\hat{y} = g(z)$, after any required decoding. The engineering objective is to reduce the byte cost $B(z)$ while retaining task quality Q :

$$\max_{f,g} Q(g(f(x)), y) \quad \text{subject to} \quad B(f(x)) \ll B(x). \quad (1.1)$$

Reconstruction fidelity is not the primary objective in Equation 1.1. A representation may reconstruct common within-class variation accurately yet remove small directions that distinguish two activities. Conversely, a representation with visible numerical error may retain the information needed by a classifier. The study therefore reports test-set Macro-F1 as its principal outcome and treats reconstruction root mean square error (RMSE) and PCA explained variance as supporting diagnostics.

1.3 Research Questions

The project addresses four research questions:

- RQ1:** How much can lossless and low-precision numerical encodings reduce the per-record feature payload without degrading gateway classification?
- RQ2:** How do linear PCA and nonlinear autoencoder representations change the relationship between transmitted bytes and Macro-F1?
- RQ3:** At matched latent dimensions, does a nonlinear autoencoder preserve activity information more effectively than PCA?
- RQ4:** Which compression family is appropriate for different accuracy and bandwidth targets, and can the recommended route be executed in a gateway deployment format?

These questions deliberately avoid claiming that there is a universally best compression algorithm. A method is useful only relative to a deployment target. A fourfold payload reduction with almost unchanged inference may be preferable to a seventyfold reduction with an eight-point quality loss in a safety-related application; the reverse can be rational for a low-rate monitoring link. The comparison must therefore expose the entire quality–communication frontier.

1.4 Scope and Assumptions

The simulated endpoint is assumed to have already computed the UCI HAR feature vector. The report does not measure the cost of converting raw accelerometer and gyroscope windows into those features. Training, PCA fitting, AE training and int8 calibration occur offline using the training partition. The endpoint stores the selected encoder or numerical scale metadata; the gateway stores the corresponding decoder where required and the activity classifier. The study measures feature-payload bytes and predictive quality. It does not claim radio energy, battery lifetime, packet-level latency or formal privacy guarantees.

The principal gateway classifier is logistic regression. This choice is methodological rather than merely convenient. The dataset already contains a large engineered representation, so a linear decision layer is a strong and interpretable reference. Keeping the classifier family fixed reduces the risk that changes in quality are caused by unrelated model tuning. A random forest is used only as an initial baseline check. For PCA and AE representations, a new classifier is fitted in each latent space because a gateway should learn the decision boundary appropriate to the actual transmitted representation.

1.5 Contributions

The report makes four practical contributions. First, it establishes an explicit endpoint–gateway accounting boundary and reports bytes per record rather than using “compression” as an undifferentiated term. Second, it compares four families—lossless byte coding, numerical quantisation, linear dimensionality reduction and nonlinear learned representation—under one subject-disjoint protocol. Third, PCA and AE are evaluated at identical dimensions of 128, 64, 32, 16 and 8, which makes their crossover interpretable. Fourth, the recommended int8 data path is checked with exported TorchScript gateway artifacts so that communication results are not disconnected from an executable deployment path.

1.6 Report Organisation

Chapter 2 develops the literature and distinguishes relevant compression families. Chapter 3 justifies the experimental design, dataset, models, metrics and reproducibility controls. Chapter 4 presents the numerical findings and explains every major table and figure. Chapter 5 interprets the research questions, engineering trade-offs, privacy implications and threats to validity. The final chapter summarises the answer and identifies extensions that require real hardware or task-aware training.

Chapter 2

Background and Literature Review

2.1 From Cloud Computing to Edge Intelligence

Cloud computing centralises elastic computation and storage, which makes it a natural location for model training and large-scale analytics. IoT applications, however, create a different physical relationship between data and compute: the observations originate at large numbers of geographically distributed devices. Shi et al. define edge computing as enabling computation at the network edge and argue that proximity can improve response time, reduce traffic and support privacy-sensitive services [1]. Satyanarayanan similarly describes edge resources as a way to place substantial computation one network hop from a mobile device rather than many variable wide-area hops away [2]. These works do not imply that the cloud disappears; they motivate a hierarchy in which endpoints, gateways and remote infrastructure handle different parts of an application.

Deng et al. organise the subsequent intersection of machine learning and edge computing under the term *edge intelligence* [3]. Their distinction between “AI on edge” and “AI for edge” is useful here. AI for edge uses learning to schedule, cache or manage network resources. AI on edge places inference or training tasks on end devices and nearby servers. The present project is an AI-on-edge study: a fixed activity classifier runs at a gateway, and the design variable is the representation sent by the endpoint. This scope excludes resource scheduling and federated optimisation, even though both could surround the system in a larger deployment.

An IoT gateway is an important middle tier because it usually has more stable power, memory and compute than an embedded sensor while remaining close to the data source. A gateway can fuse multiple devices, enforce local policy and send only

decisions or summaries onward. Local inference can also continue during a cloud outage. The benefit is conditional: if endpoint preprocessing or a learned encoder consumes more energy than the radio traffic it saves, moving work toward the edge can be counterproductive. Consequently, edge inference should be evaluated as a coupled communication–computation decision rather than a slogan.

2.2 Collaborative and Split Inference

Several systems studies distribute a neural network across device, edge and cloud resources. Neurosurgeon profiles neural-network layers and selects a split that reduces latency or mobile energy under the current device and network conditions [6]. Teerapittayanon et al. map sections of a distributed deep neural network onto end, edge and cloud tiers and show that local exits and feature transfer can reduce communication while retaining useful predictions [7]. These studies establish that the position of an inference boundary matters because intermediate activations vary substantially in size and computational cost.

Feature compression makes the split-point problem more explicit. Choi and Bajic study lossy compression of intermediate features for collaborative object detection and show that task accuracy, not visual reconstruction alone, must govern the transmitted representation [8]. BottleNet++ jointly learns feature compression and a communication-channel model for device–edge co-inference, reporting large reductions under its evaluated image tasks and channel assumptions [9]. These works are closer to task-aware communication than conventional file compression: the feature is useful because of the prediction it enables downstream.

The current project differs from those split-DNN systems in two ways. First, the endpoint is not executing the early layers of the same gateway classifier. It is encoding a fixed sensor feature vector, and the gateway learns a classifier in the received space. Second, the project intentionally compares conventional families—zlib, quantisation, PCA and a reconstruction-trained AE—before moving to a jointly optimised task-aware encoder. This creates an auditable baseline for future collaborative inference and prevents an advanced method from being judged without simple alternatives.

2.3 Human Activity Recognition at the Edge

Sensor-based HAR converts inertial or ambient measurements into labels such as walking, sitting and standing. The UCI HAR benchmark derives time- and frequency-domain statistics from fixed accelerometer and gyroscope windows and uses a participant-disjoint train/test partition [4, 5]. Because neighbouring windows from the same person can be highly correlated, a subject-disjoint split is more credible than randomly mixing individual windows between training and testing. It asks whether the model generalises to unseen people rather than memorising person-specific signal structure.

HAR is well suited to an edge study for operational and ethical reasons. A local gateway may need to produce a prompt response for fall detection, rehabilitation feedback or context-aware assistance. Continuous raw motion traces can also reveal routines and behavioural attributes. Keeping inference near the user can reduce unnecessary cloud transfer, although local processing alone is not a formal privacy guarantee. The UCI benchmark contains engineered feature vectors, so it isolates the transmission problem after signal processing. Results therefore apply to feature-payload compression rather than the complete energy cost of sampling and feature extraction.

The HAR literature contains increasingly complex deep architectures, but model novelty is not the central issue in this report. An elaborate temporal network would mix improvements from feature learning with changes caused by compression. Using the established engineered features and a fixed lightweight classifier creates a clearer causal comparison. It also reflects a realistic gateway that must remain maintainable and execute without an accelerator.

2.4 A Taxonomy of Compression Families

The word *compression* is used for several technically different interventions. Table 2.1 separates them by the object being reduced, loss characteristics and deployment requirements. The distinction is necessary because two methods can have the same nominal compression ratio while imposing very different endpoint computation and gateway compatibility.

CHAPTER 2. BACKGROUND AND LITERATURE REVIEW

Table 2.1: Compression families relevant to the endpoint–gateway data path.

Family	Object reduced	Lossy?	Main engineering characteristic
Lossless byte coding	Serialised record	No	Exact recovery and classifier reuse; gain depends on byte-level redundancy and batching.
Numerical quantisation	Precision per feature	Yes	Fixed and predictable payload; calibration metadata and possible clipping.
Linear representation	Feature dimension	Yes	Transparent matrix projection; decoder optional when classifier is trained in latent space.
Learned representation	Feature dimension	Yes	Nonlinear capacity and flexible objective; requires model storage and endpoint execution.
Model compression	Gateway artifact	Usually	Reduces memory or arithmetic, but does not itself reduce sensor-message bytes.

2.4.1 Lossless Byte Compression

Lossless compression finds statistical redundancy in a serialised byte stream. DEFLATE, exposed through zlib, combines dictionary matching with entropy coding. After decompression the gateway obtains exactly the original float32 record; a deterministic classifier must therefore make the same prediction. This property makes lossless coding an essential control. It demonstrates how much can be saved without changing the inference problem at all.

The limitation is granularity. A single record of 561 preprocessed floating-point values is short and may contain little repeated byte structure. Compressing a large batch can improve the coding ratio by exposing cross-record redundancy, but batching delays an inference until enough observations accumulate. The present experiment compresses each record independently to match a low-latency gateway path. Its lossless result should not be extrapolated to archival compression of a whole dataset.

2.4.2 Numerical Quantisation

Quantisation replaces high-precision real values with a smaller discrete set. The technique is widely studied for neural-network weights and activations because inte-

CHAPTER 2. BACKGROUND AND LITERATURE REVIEW

ger arithmetic and compact parameters can lower model storage and, on appropriate hardware, computation cost [10, 11]. In this report the target is different: the quantised object is the sensor message. Float16 rounding halves the bytes of every feature with no learned calibration. Signed int8 uses one byte per feature and a training-derived scale.

For feature-wise symmetric int8, feature j has calibration scale

$$s_j = \frac{\max_{x \in \mathcal{D}_{train}} |x_j|}{127}, \quad q_j = \text{clip} \left(\text{round} \left(\frac{x_j}{s_j} \right), -127, 127 \right), \quad (2.1)$$

and the gateway reconstructs $\tilde{x}_j = s_j q_j$. Feature-wise scaling is preferred to one global scale because the 561 engineered quantities have different ranges. Calibration on training data avoids test leakage. The scale vector is provisioning metadata shared with a model version rather than repeated in every message. Quantisation is attractive when a predictable fixed-width payload and a simple endpoint are more valuable than the largest possible ratio.

2.4.3 Principal Component Analysis

PCA projects centred observations onto orthogonal directions of decreasing training-set variance [12]. For standardised data matrix X , the rank- k representation is $Z = XW_k$, where columns of W_k are the first k right singular vectors. A k -dimensional float32 code occupies $4k$ bytes. PCA is deterministic after fitting, has a well-defined inverse projection and can be implemented as a matrix–vector product. Explained variance offers a useful description of numerical information retained.

Variance, however, is not identical to discriminative information. A low-variance direction may distinguish sitting from standing, while a high-variance direction may represent subject-specific motion amplitude. PCA is unsupervised and has no access to the label. This is why downstream Macro-F1 must be measured directly and why an explained-variance threshold alone is an inadequate selection rule.

2.4.4 Autoencoder Representations

An autoencoder learns an encoder f_θ and decoder h_ϕ by minimising a reconstruction objective, here mean squared error:

$$\min_{\theta, \phi} \frac{1}{n} \sum_{i=1}^n \|x_i - h_{\phi}(f_{\theta}(x_i))\|_2^2. \quad (2.2)$$

The bottleneck $z = f_{\theta}(x)$ becomes the transmitted code. Hinton and Salakhutdinov demonstrated that nonlinear networks can learn compact representations that differ from linear PCA [13]. Nonlinearity can preserve curved structure or interactions when a bottleneck is narrow, but it introduces training variance, hyperparameters, storage and endpoint compute. Moreover, a reconstruction-trained AE remains task-agnostic: it can allocate capacity to detail irrelevant to the class label.

The present architecture is intentionally modest and symmetric. Standardised 561-dimensional inputs pass through 128 ReLU units, a bottleneck of dimension k , another 128 ReLU units and a 561-dimensional output. This supplies a fair, understandable nonlinear baseline rather than an exhaustive neural architecture search. The classifier is fitted directly on the latent code; decoding is needed for the reconstruction diagnostic but not for normal gateway classification.

2.4.5 Task-Aware and Jointly Learned Compression

Task-aware compression optimises the transmitted representation for a downstream loss instead of, or in addition to, signal reconstruction. Collaborative inference studies show why this can be powerful: a representation need not retain every detail if it preserves what the task requires [8, 9]. Joint training can also account for channel noise or a particular split point. The cost is specificity. A code trained only for six activity labels may be less useful for a second gateway task, and comparison becomes sensitive to classifier capacity and loss weighting.

This family defines the clearest extension of the project. It is not included in the current experiment because the objective is first to establish how far standard, reproducible baselines can go. The AE result is therefore not presented as the state of the art in semantic communication; it is a conventional learned representation against which a future task-aware encoder can be measured.

2.5 Model Compression Is a Separate Axis

Post-training model quantisation reduces a deployed classifier’s parameters and may enable efficient integer kernels [10]. It is conceptually separate from Equation 2.1: one quantises the gateway model, the other quantises the feature payload. A production pipeline may use both. The report therefore includes a TorchScript dynamic-int8 classifier as a deployment proxy but never adds its model-size saving to the message compression ratio. This separation prevents a common reporting error in which wire bytes and artifact bytes are combined into a single number without a time horizon.

2.6 Synthesis and Research Gap

The literature establishes three relevant facts. First, computation close to the data source can reduce dependency on a remote cloud. Second, the placement of an inference boundary and the size of the transmitted representation are coupled. Third, low precision and learned features can reduce storage or communication, but their benefit must be evaluated using the downstream task.

What an engineer still needs is a comparable, application-level view of simple compression families under one protocol. Existing edge surveys motivate local inference; split-computing work often uses images and jointly trained deep models; and model-compression work usually measures parameters or operations. This study fills a smaller empirical gap for engineered IoT sensor features: it reports actual per-record payloads and subject-disjoint HAR quality for lossless coding, two numerical precisions, PCA and AEs at matched dimensions. The study does not claim a new algorithm. Its value lies in a controlled frontier, a transparent accounting boundary and the interpretation required to choose an operating point.

Chapter 3

Experimental Methodology and Implementation

3.1 Experimental Design

The experiment is designed as a sequence of increasingly aggressive changes to one endpoint–gateway data path. Phase 1 establishes the raw classifier and a loss-less transmission reference. Phase 2 changes numerical precision while retaining 561 features. Phase 3 projects the vector with PCA. Phase 4 replaces the linear projection with an autoencoder bottleneck. Phase 5 combines all results into one quality–payload frontier, and Phase 6 exports a gateway classifier to test the recommended numerical path. This phased design makes each result independently inspectable and supports incremental reproduction.

The controlled independent variable is the representation sent by the endpoint. The principal dependent variables are payload bytes per record and test Macro–F1. The official training and test partitions, class labels and data preprocessing convention are held constant. For lossy latent representations a classifier is retrained in each space, because applying the raw-space classifier after reconstruction would unnecessarily entangle representation quality with decoder error. The test partition is used once for final reporting, not for fitting encoders, choosing scales or stopping AE training.

Table 3.1: Dataset and simulated deployment mapping.

Item	Configuration
Dataset	UCI Human Activity Recognition Using Smartphones
Training/test records	7,352 / 2,947; official subject-disjoint split
Input	561 engineered accelerometer and gyroscope features
Output	Six activities of daily living
IoT endpoint	Smartphone or embedded sensor node holding one feature vector
Communication link	Logical endpoint-to-local-gateway hop
Gateway	Computer that decodes where necessary and predicts the activity
Training	Offline, using training data only

3.2 Dataset and System Mapping

The UCI HAR dataset contains 7,352 training records and 2,947 test records. Each record has 561 engineered features and one of six labels: walking, walking upstairs, walking downstairs, sitting, standing and laying. The official split is organised by participant, so subjects in the test set do not occur in the training set [4, 5]. Table 3.1 summarises the experimental unit.

The mapping requires a precise caveat. UCI HAR supplies features rather than raw streaming samples. The reported 2,244-byte baseline equals 561×4 bytes for float32 values; it does not include the samples from which those features were derived. This boundary is useful because every evaluated method starts with an identical record. It is not a complete smartphone energy model. In a physical system, feature extraction, timestamps, device identifiers, encryption and packet headers must be accounted for separately.

3.3 Baseline Classifier Selection

Two conventional classifiers were evaluated on the raw training features: standardised multinomial logistic regression and a 300-tree random forest. The logistic pipeline uses a **StandardScaler**, regularisation parameter $C = 1.0$ and a maximum of 3,000 optimisation iterations. Logistic regression was retained because it

produced the stronger validation of the engineered feature space and has a small, transparent deployment footprint. The purpose is not to establish the best possible HAR model. It is to create a stable gateway decision function against which input representations can be compared.

For raw, float16 and int8 inputs, the fitted raw-space pipeline remains fixed. This isolates the effect of rounding and dequantisation. For PCA and AE codes, a separate standardised logistic classifier is fitted for each k . Retraining is methodologically justified because the transmitted vector is the operational input to the gateway. A classifier trained in that space can use all surviving discriminative structure without requiring reconstruction to 561 dimensions.

3.4 Transmission Baselines

3.4.1 Raw float32

The uncompressed reference is a contiguous vector of 561 IEEE-754 float32 values:

$$B_{raw} = 561 \times 4 = 2,244 \text{ bytes per record.} \quad (3.1)$$

Using fixed-width arithmetic rather than an on-disk text or NumPy container keeps the metric tied to the message representation. File headers and array meta-data would otherwise dominate small examples and differ between software libraries.

3.4.2 Record-wise zlib

Each float32 vector is serialised into 2,244 raw bytes and compressed separately using zlib. Mean compressed bytes across all test records are reported. Decoding is verified to reproduce the original byte sequence. Independent records reflect a gateway that produces a decision for every window. A batched archive could achieve a different ratio and is outside this latency boundary.

3.5 Numerical Quantisation

Float16 is implemented as a float32-to-float16 cast at the endpoint and a float16-to-float32 cast at the gateway. Its payload is exactly $561 \times 2 = 1,122$ bytes. No additional scale metadata is required. The raw classifier then receives the rounded float32 reconstruction. Input error and changed predictions are recorded to detect cases where an apparently unchanged aggregate score hides individual decision changes.

The int8 path uses Equation 2.1. For every feature, the maximum absolute training value determines a symmetric scale. A zero-range feature receives a safe non-zero fallback scale. Endpoint values are rounded and clipped to $[-127, 127]$; the gateway multiplies the received integer by the agreed scale. The message is 561 bytes. The 561 float32 scales occupy 2,244 bytes as one-time provisioning metadata. They are not charged per observation because the same versioned vector is reused across records, just as classifier coefficients are installed rather than resent with every prediction request.

3.6 PCA Representation

PCA is fitted after standardising the training features. Bottleneck dimensions $k \in \{128, 64, 32, 16, 8\}$ are selected to span modest through severe compression. The endpoint applies the fitted scaler and projection, transmitting k float32 scores. The gateway classifier is trained directly on those scores. The per-record payload and compression ratio are

$$B_{PCA}(k) = 4k, \quad R_{PCA}(k) = \frac{2244}{4k}. \quad (3.2)$$

The implementation records cumulative explained variance, reconstruction RMSE, accuracy and Macro-F1. Explained variance describes the projection but is not a selection target. Training-only fitting is essential: fitting the scaler or PCA basis to the concatenated dataset would allow the test distribution to influence the representation.

3.7 Autoencoder Representation

The AE operates on the same standardised training features. Its architecture is

$$561 \rightarrow 128 \text{ ReLU} \rightarrow k \rightarrow 128 \text{ ReLU} \rightarrow 561. \quad (3.3)$$

For each $k \in \{128, 64, 32, 16, 8\}$, 15% of the official training partition is reserved as a stratified validation subset. Optimisation uses Adam, learning rate 10^{-3} , batch size 256, MSE reconstruction loss and at most 60 epochs. Early stopping uses validation loss with patience 10, and the best validation checkpoint is restored. Random seeds are fixed to 5105 for Python, NumPy and PyTorch where applicable. The gateway logistic classifier is fitted on training latent codes.

Using the same k values makes PCA and AE payloads identical, so the comparison cannot be explained by different byte budgets. The AE decoder is retained to calculate reconstruction RMSE but is not required for activity inference. The encoder is an endpoint artifact, and its storage and computation are explicitly considered in the discussion. This prevents the 32-byte AE-8 message from being treated as cost-free.

3.8 Metrics and Decision Rules

Accuracy is reported for familiarity, but Macro-F1 is primary because it gives equal weight to each class. For class c , precision and recall are

$$P_c = \frac{TP_c}{TP_c + FP_c}, \quad R_c = \frac{TP_c}{TP_c + FN_c}, \quad (3.4)$$

and $F1_c = 2P_cR_c/(P_c + R_c)$. Macro-F1 is the unweighted mean over the six classes. Communication is reported as payload bytes and compression ratio $R = B_{raw}/B_{method}$. Percentage-point changes are used for task quality; relative ratios are reserved for bytes.

Three post-hoc application policies translate the frontier into decisions:

1. **near-baseline quality**: select the smallest payload within 0.005 absolute Macro-F1 of the raw score;
2. **balanced**: select the smallest tested payload with Macro-F1 at least 0.90;

3. **extreme compression**: select the smallest payload with Macro-F1 at least 0.85.

These thresholds are illustrative and declared before the recommendation is interpreted. They are not universal safety requirements.

3.9 Deployment Validation

The fitted logistic regression is converted to an equivalent PyTorch linear module and exported as TorchScript. Its predictions are checked against the scikit-learn reference. Dynamic int8 quantisation is then applied to the linear layer, producing a smaller gateway artifact. Two validations are distinguished: model-only dynamic quantisation on raw float32 features, and the end-to-end route in which the endpoint sends int8 features, the gateway dequantises them and a dynamic-int8 classifier predicts the label.

Sequential desktop latency is recorded as a compatibility diagnostic, not a hardware claim. A tiny linear model can be slower after dynamic quantisation if dispatch and conversion overhead exceed arithmetic savings. Actual gateway latency and energy require a specified processor, runtime, thread policy and power-measurement method.

3.10 Reproducibility and Leakage Controls

All experiments are implemented as phase-specific Python entry points. Each phase writes machine-readable CSV or JSON metrics and publication-ready plots. Dataset loading validates expected shapes, labels and split files. The official test set is never merged into training. Standardisers, calibration scales, PCA bases and AEs are fitted on training data; AE early stopping uses only its training-derived validation subset. Fixed dimensions and random seeds are recorded in source.

The scripts are `run_phase1.py` through `run_phase6_deployment.py`. Their outputs are stored under `code/outputs/phase1` to `code/outputs/phase6`, with the unified comparison under `code/outputs/final`. Appendix A provides exact execution order, environment files, artifacts and validation checks.

Chapter 4

Experimental Results

4.1 Raw and Lossless Baselines

The standardised logistic-regression pipeline achieved 95.453% accuracy and 95.443% Macro-F1 on the 2,947-record official test partition. This score is the quality reference for all subsequent compression methods. The baseline also establishes that the engineered UCI features support a strong lightweight gateway classifier; improvements or degradations caused by encoding are therefore not hidden by an unusably weak decision model.

Record-wise zlib reduced the 2,244-byte float32 payload to a mean of 1,973.37 bytes, a compression ratio of only $1.137\times$. Because the transform is lossless, predictions and quality are exactly unchanged. The modest ratio is important rather than disappointing: it shows that generic lossless coding has limited redundancy to exploit inside one short, preprocessed floating-point record. It establishes why lossy numerical or representational changes are needed for substantial communication reduction.

Table 4.1 uses mean compressed bytes for zlib because records produce variable-length outputs. The ratio is computed from the mean payload, not by averaging individual ratios, which avoids overweighting unusually small records. No protocol header is included in either row.

4.2 Numerical Quantisation

Table 4.2 shows that the 561-dimensional representation tolerates substantial precision reduction. Float16 halves the payload to 1,122 bytes and produces the

CHAPTER 4. EXPERIMENTAL RESULTS

Table 4.1: Raw and record-wise lossless baselines.

Method	Bytes/record	Ratio	Accuracy	Macro-F1
Raw float32	2,244.00	1.000×	95.453%	95.443%
zlib float32	1,973.37	1.137×	95.453%	95.443%

Table 4.2: Numerical-precision results with the raw-space gateway classifier.

Representation	Bytes	Ratio	Accuracy	Macro-F1
float32	2,244	1.00×	95.453%	95.443%
float16	1,122	2.00×	95.453%	95.443%
Symmetric int8	561	4.00×	95.521%	95.502%

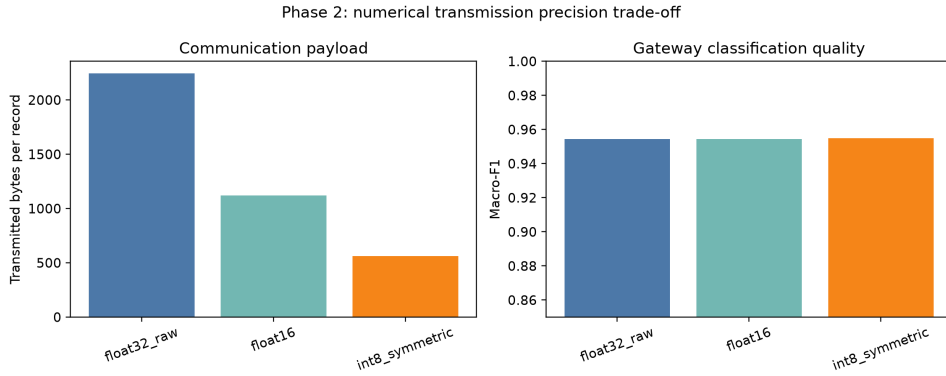


Figure 4.1: Quantisation trade-off. Moving from float32 to float16 and int8 gives deterministic fixed-width payload reductions while the Macro-F1 markers remain effectively level. The figure therefore identifies int8 as a Pareto improvement over the raw message for this dataset and classifier.

same aggregate Macro-F1 to the reported precision. The converted feature values are not bit-identical to float32, but none of the rounding differences crosses a relevant decision boundary in aggregate.

Feature-wise symmetric int8 reduces the payload to 561 bytes, exactly one quarter of the raw feature bytes. Its Macro-F1 is 95.502%, 0.059 percentage points above the raw score. This small increase should be interpreted as numerical variation or a mild regularising effect, not as evidence that quantisation improves the underlying task. The important conclusion is non-inferiority at a fourfold communication reduction.

Figure 4.1 should not be read as a general claim that every sensor feature can be quantised safely. The scales were calibrated on the training distribution, the test

Table 4.3: PCA payload, explained variance and downstream quality.

k	Bytes	Ratio	Explained variance	Macro-F1
128	512	$4.38\times$	96.98%	93.026%
64	256	$8.77\times$	90.23%	91.949%
32	128	$17.53\times$	82.50%	88.931%
16	64	$35.06\times$	75.12%	85.870%
8	32	$70.13\times$	68.85%	83.548%

range is similar and the logistic decision surface is robust to the observed rounding. A new device population or sensor fault could create clipping, so deployed systems should monitor scale violations.

4.3 PCA Results

PCA provides a smooth but costly quality–dimension trade-off. At 128 components the payload is 512 bytes, slightly smaller than int8, and Macro-F1 is 93.026%. Reducing the code to 64, 32, 16 and 8 components yields 91.949%, 88.931%, 85.870% and 83.548% respectively. The eight-component code is only 32 bytes, but it loses 11.895 percentage points relative to raw.

The first row illustrates why explained variance is not a task guarantee. Although PCA-128 retains nearly 97% of standardised training variance, its Macro-F1 is about 2.42 percentage points below raw. Activity discrimination can depend on directions that contribute relatively little total variance. Conversely, the 64-component code remains above 91.9% Macro-F1 even after almost 89% of payload bytes have been removed, showing that the original feature vector is highly redundant for this classifier.

Figure 4.2 makes the absence of a single optimal PCA dimension visible. The appropriate point depends on the application’s threshold. PCA-64 is viable for a target around 0.90 Macro-F1, while PCA-8 is only reasonable if a much lower quality floor is acceptable. The curve is also a linear baseline against which the AE must be judged at exactly the same byte values.

CHAPTER 4. EXPERIMENTAL RESULTS

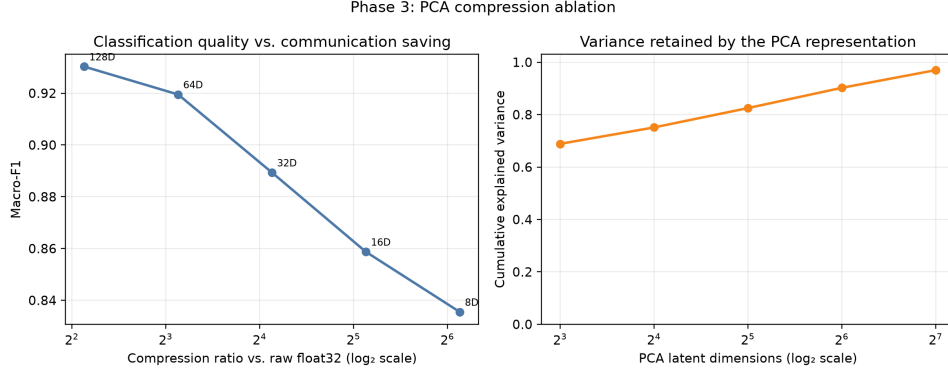


Figure 4.2: PCA quality–compression curve across matched bottleneck dimensions. Each move to the right reduces payload but also removes discriminative information; there is no broad plateau comparable with numerical int8.

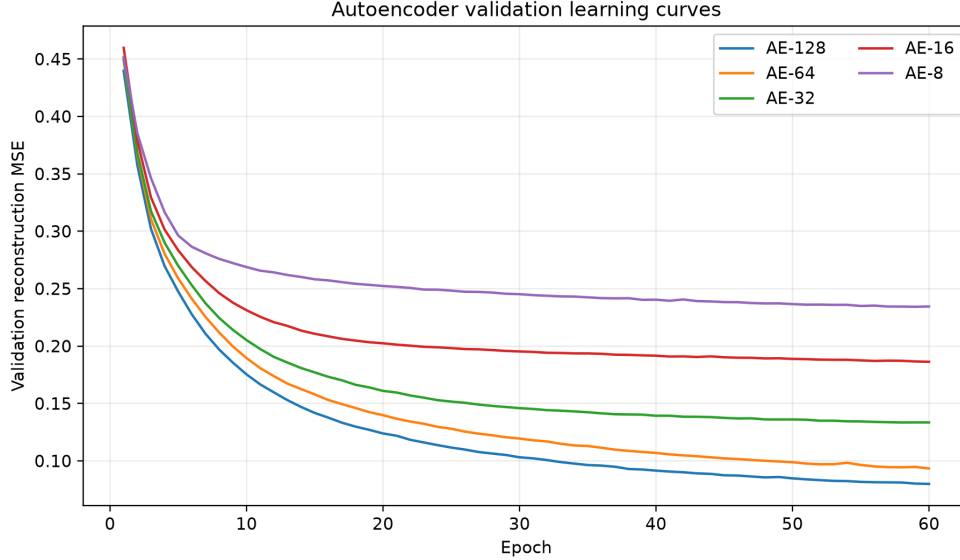


Figure 4.3: Training and validation reconstruction loss for AE bottlenecks. The curves support optimisation stability and checkpoint selection; they do not by themselves establish that the latent codes preserve activity labels.

4.4 Autoencoder Results

The AE training curves converged under early stopping for all tested bottlenecks. The latent code becomes progressively harder to reconstruct as k decreases, which is expected. More important is the non-monotonic relationship between reconstruction and classification: the representation that reconstructs most accurately is not automatically the most efficient communication choice.

At 128 components the AE reaches 91.785% Macro-F1, lower than PCA-128.

Table 4.4: Autoencoder latent-space results.

k	Bytes	Ratio	Reconstruction RMSE	Macro-F1
128	512	$4.38\times$	0.0868	91.785%
64	256	$8.77\times$	0.0943	91.215%
32	128	$17.53\times$	0.1091	90.539%
16	64	$35.06\times$	0.1280	88.196%
8	32	$70.13\times$	0.1420	86.773%

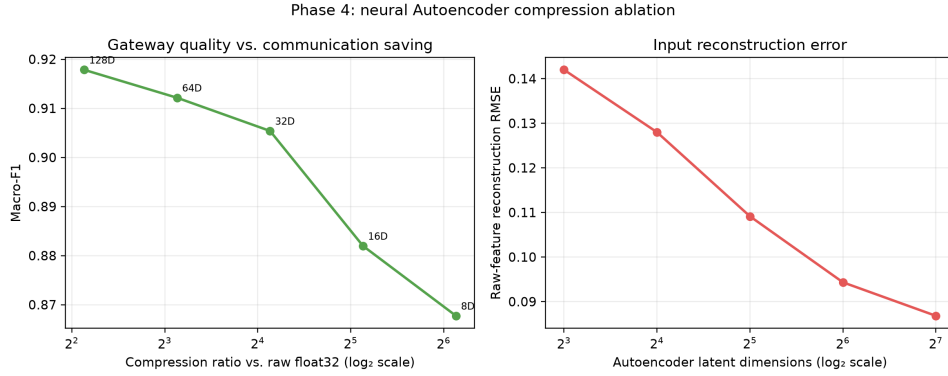


Figure 4.4: Autoencoder quality–compression relationship. Quality declines with bottleneck width, but the narrowest codes retain more Macro-F1 than the matched PCA alternatives shown later in Figure 4.5.

The gap narrows at 64 components and reverses at 32. AE-32 retains 90.539% Macro-F1 in 128 bytes, while PCA-32 retains 88.931%. The nonlinear advantage grows at 16 and 8 components. AE-8 reaches 86.773%, 3.225 percentage points above PCA-8 at the same 32-byte payload.

The result does not show that AEs dominate PCA. It shows a crossover: PCA is better for moderate representation compression, while this AE is better when the bottleneck is severe. Architecture, training objective and random seed can affect the crossover. The observation is therefore phrased as a result of the evaluated configuration, not a universal theorem about nonlinear representation learning.

4.5 Unified Compression-Family Comparison

Table 4.5 places all data-path methods under one accounting rule. The table exposes three distinct regions. Lossless zlib produces a small exact saving. Numerical float16 and int8 produce large fixed-width savings without an observed quality cost.

CHAPTER 4. EXPERIMENTAL RESULTS

Dimensional representations reach much larger ratios, but their quality decreases with the available code width.

Table 4.5: Unified endpoint-to-gateway comparison.

Method	Family	Bytes	Ratio	Macro-F1
Raw float32	Raw	2,244	$1.00\times$	95.443%
zlib float32	Lossless	1,973.37	$1.14\times$	95.443%
float16	Quantisation	1,122	$2.00\times$	95.443%
Symmetric int8	Quantisation	561	$4.00\times$	95.502%
PCA-128	Linear latent	512	$4.38\times$	93.026%
AE-128	Learned latent	512	$4.38\times$	91.785%
PCA-64	Linear latent	256	$8.77\times$	91.949%
AE-64	Learned latent	256	$8.77\times$	91.215%
PCA-32	Linear latent	128	$17.53\times$	88.931%
AE-32	Learned latent	128	$17.53\times$	90.539%
PCA-16	Linear latent	64	$35.06\times$	85.870%
AE-16	Learned latent	64	$35.06\times$	88.196%
PCA-8	Linear latent	32	$70.13\times$	83.548%
AE-8	Learned latent	32	$70.13\times$	86.773%

Figure 4.5 is the central empirical result. The int8 marker is a clear elbow: methods with slightly smaller payloads already lose multiple quality points. At the other extreme, PCA and AE share the same horizontal coordinates at matched k , so vertical separation directly reflects label preservation. AE-32, AE-16 and AE-8 sit above their PCA counterparts and therefore dominate them for those exact payload budgets. PCA-128 and PCA-64 sit above the matched AE points and remain simpler to deploy.

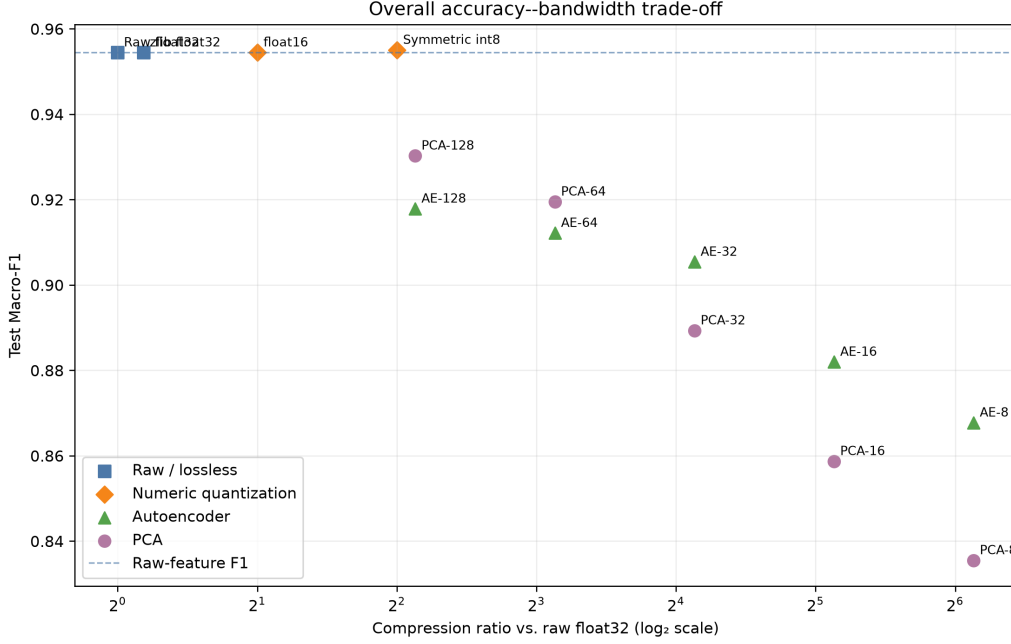


Figure 4.5: Unified payload–Macro–F1 frontier. The upper-left region is preferable. Int8 is the near-lossless operating point; AE-32 and AE-8 extend the frontier when the permitted message is 128 or 32 bytes respectively.

Table 4.6: Selections for illustrative application targets.

Target	Rule	Method	Bytes	Macro–F1
Near baseline	Within 0.005 of raw	int8	561	95.502%
Balanced	At least 0.90	AE-32	128	90.539%
Extreme	At least 0.85	AE-8	32	86.773%

4.6 Application-Oriented Recommendations

Applying the declared policies gives the three operating points in Table 4.6. Symmetric int8 is the smallest payload within 0.005 Macro–F1 of raw. AE-32 is the smallest tested payload above 0.90, and AE-8 is the smallest tested payload above 0.85. These are selections from the evaluated grid; a dimension between the tested values might create another feasible point.

4.7 Gateway Deployment Proxy

The exported float32 TorchScript classifier occupies 22,583 bytes and reproduces the scikit-learn predictions exactly. Dynamic int8 reduces the artifact to 16,596

CHAPTER 4. EXPERIMENTAL RESULTS

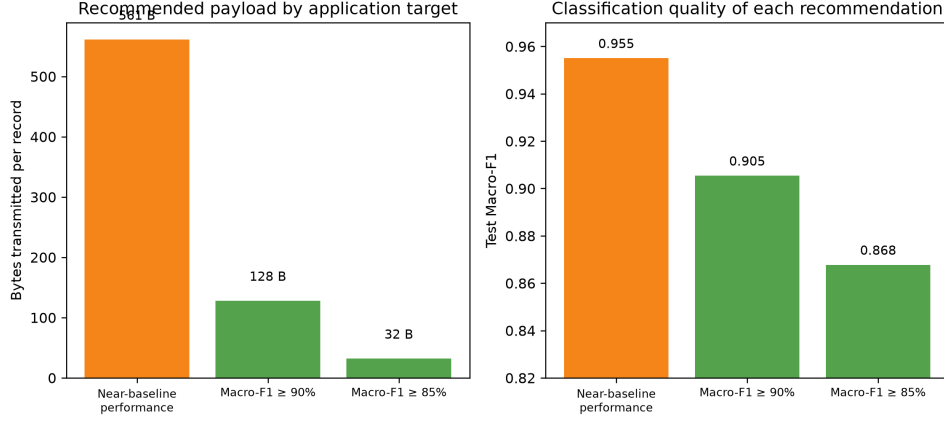


Figure 4.6: Recommended points for three bandwidth-quality policies. The bars do not imply equal endpoint cost: the AE points require a neural encoder, whereas int8 requires only calibrated scaling, rounding and clipping.

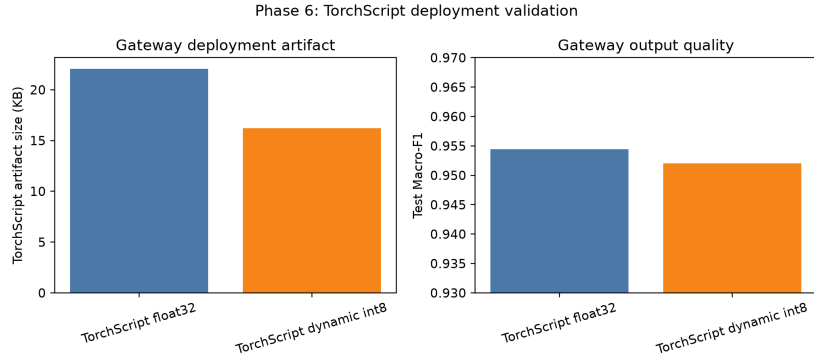


Figure 4.7: Gateway deployment comparison. Dynamic int8 reduces classifier storage while retaining high test quality. This model-artifact reduction is reported separately from the $4\times$ feature-message reduction.

bytes, a 26.5% decrease, with 95.201% Macro-F1. It changes 43 of 2,947 predictions relative to the float artifact. When endpoint feature int8, dequantisation and the dynamic-int8 gateway model are combined, the complete path achieves 95.035% Macro-F1 with 561 transmitted bytes per record.

Sequential desktop latency was 0.00887 ms per record for float TorchScript and 0.02595 ms for dynamic int8. The quantised model is therefore smaller but slower in this particular microbenchmark. A tiny linear layer performs too little arithmetic to amortise quantisation and dispatch overhead. The finding validates executability and cautions against assuming that a smaller model is automatically faster; it is not a latency conclusion for ARM, Raspberry Pi or microcontroller hardware.

Chapter 5

Discussion

5.1 Answers to the Research Questions

5.1.1 RQ1: Lossless and Low-Precision Transmission

Record-wise lossless compression is not sufficient for this feature representation. Its $1.14\times$ ratio leaves almost 88% of raw bytes on the link. The result is consistent with the limited repetition available inside a single sequence of already processed floating-point values. Lossless zlib remains valuable as a control because it guarantees identical inputs and predictions, but it does not meet a strong bandwidth-reduction objective.

Numerical quantisation is much more effective. Float16 deterministically halves the message and int8 reduces it by 75%, with no material loss in aggregate quality. The answer to RQ1 is therefore that low precision, rather than generic record-wise coding, supplies the best near-baseline trade-off in this experiment. The result depends on feature-wise calibration and on the absence of severe out-of-range test observations; it should be revalidated after a sensor or population change.

5.1.2 RQ2: Dimensionality and the Quality Frontier

PCA and AE produce ratios from $4.38\times$ to $70.13\times$, far beyond the fixed fourfold limit of one-byte-per-feature int8. Their price is a descending quality curve. This shows why reporting only a compression ratio is incomplete. AE-8 and PCA-8 have the same 32-byte payload but differ by more than three Macro-F1 percentage points, while int8 uses 561 bytes and retains about eight more points than either.

RQ2 is answered by a frontier, not a single number: representation compression becomes useful when link constraints justify a measurable task loss and endpoint transformation cost.

5.1.3 RQ3: PCA versus Autoencoder

PCA is stronger at 128 and 64 dimensions; AE is stronger at 32, 16 and 8. The matched-byte design makes this crossover meaningful. A plausible explanation is that broad variance structure is efficient for PCA at moderate dimensions, whereas a nonlinear bottleneck preserves some interaction or curved structure after the space becomes very narrow. The experiment does not prove that latent geometry is the mechanism, and it does not show that any AE will outperform any PCA. It establishes the observed ordering for one architecture, seed policy and sensor task.

This negative-and-positive result is more useful than a simple claim that learned compression is better. Nonlinear capacity creates optimisation and deployment cost and is not automatically converted into task quality. An engineer with a 256-byte budget should prefer PCA-64 in these results; an engineer with a 128-byte budget and a 0.90 quality floor should prefer AE-32.

5.1.4 RQ4: Recommended Deployment Points

Symmetric feature-wise int8 is the default recommendation. It offers a large, predictable saving, nearly unchanged inference, simple endpoint arithmetic and reuse of the raw-space classifier. AE-32 is the balanced choice at 128 bytes, and AE-8 is an extreme 32-byte option if 86.77% Macro-F1 is acceptable. The TorchScript experiment confirms that the int8 feature route can feed an exported gateway model, while also showing that dynamic model quantisation should be judged separately from message quantisation.

5.2 Systematic Comparison of Compression Families

Table 5.1 reveals why payload is only one decision variable. zlib and float16 are largely stateless and preserve the existing model interface. Int8 adds calibration

Table 5.1: Qualitative comparison beyond payload and Macro-F1.

Family	Endpoint compute	State to provision	Gateway change	Primary failure mode
zlib	General-purpose coder	Codec version	Exact decompression	Poor ratio or variable length
float16	Cast	Numeric convention	Cast to float32	Rounding/overflow
int8	Scale, round, clip	561 scales	Dequantisation	Range drift and clipping
PCA	Matrix-vector product	Scaler and basis	Latent classifier	Linear loss of label directions
AE	Neural encoder	Weights and scaler	Latent classifier	Model drift, compute cost, unstable out-of-distribution codes

state but remains simple. PCA and AE change the semantic interface: the gateway classifier and endpoint transform must share a version. If the encoder is updated without the classifier, identical bytes acquire a different meaning. Version negotiation, rollback and monitoring are therefore part of a representation-compression deployment.

The one-time installation cost also changes the conclusion for short deployments. The int8 scale vector is 2,244 bytes and is amortised after only a few records. A PCA-128 projection has 561×128 coefficients before packaging, and the saved AE artifacts in the experiment range from approximately 590 to 713 KB. If a device sends only a handful of records before a model update, encoder delivery may cost more than the communication it saves. In continuous monitoring, the same installation is amortised over thousands or millions of windows.

Endpoint arithmetic may trade radio energy for CPU energy. Int8 uses element-wise operations and is likely to be inexpensive on common microcontrollers. PCA uses a dense projection, and the AE uses two endpoint layers to reach its bottleneck. The present desktop encode timings are not used to make battery claims. A real selection should measure the combined energy

$$E_{total} = E_{feature} + E_{encode} + E_{radio}(B) + E_{gateway}, \quad (5.1)$$

under a specified sampling rate, radio and processor. The smallest B does not necessarily minimise E_{total} .

5.3 Interpreting the PCA–AE Crossover

At 128 dimensions PCA retains 96.98% of variance but loses about 2.42 Macro–F1 points. The divergence demonstrates that unsupervised reconstruction and label prediction answer different questions. High-variance directions can capture subject motion amplitude or other nuisance variation. A relatively low-variance direction can still separate sitting from standing. Consequently, explained variance should describe the encoder rather than serve as the acceptance metric.

At very narrow dimensions, the AE’s nonlinear mapping may allocate capacity more flexibly than a single global subspace. Yet the AE is also trained with MSE, not activity loss. Its advantage at $k \leq 32$ is therefore achieved without direct supervision and may be improved by a hybrid objective. A future task-aware model could minimise

$$\mathcal{L} = \lambda \mathcal{L}_{reconstruction} + (1 - \lambda) \mathcal{L}_{classification} + \beta \mathcal{R}_{rate}, \quad (5.2)$$

where $\mathcal{R}_{rate}$ estimates transmitted bits. Varying λ would show whether a code can support activity recognition and broader sensor reuse. The current results supply the reconstruction-only baseline such a model needs.

5.4 Payload Accounting in a Real Network

The reported bytes deliberately describe only the feature payload. A real message includes a timestamp, device identity, encoder version, integrity protection and transport headers. Fixed overhead becomes proportionally larger for a tiny code. For illustration, adding 40 bytes to every message changes the raw-versus-AE-8 ratio from $2244/32 = 70.13$ to $2284/72 = 31.72$. The saving remains substantial but is less spectacular. This example is not a measurement of Wi-Fi, BLE or MQTT; it shows why the accounting boundary must accompany every ratio.

Batching creates another trade-off. Compressing multiple records together can improve zlib and amortise headers, but it delays the oldest observation and may make loss recovery more expensive. Interactive HAR may require one prediction per window, while archival wellness analytics may tolerate batches. The chosen record-wise experiment represents the former. A packet-level follow-up should vary batch size and report latency percentiles as well as bytes.

5.5 Operational Deployment Policy

A production int8 route should provision the scale vector, classifier and schema under one immutable version. Messages should identify that version, and the gateway should reject unknown encodings rather than silently apply the wrong scale. The endpoint can count the fraction of clipped features; a rising clipping rate is an inexpensive indicator that the training range no longer describes the device. Periodic labelled audits are still required because low clipping does not guarantee stable class quality.

PCA and AE deployments need additional monitoring. Projection or encoder versions must be paired with the corresponding latent classifier. Reconstruction error can serve as a weak anomaly signal if the decoder is available, but it is not a substitute for task labels. A safe migration can run old and new encoders in shadow mode, compare gateway outputs and roll back before deleting the previous artifact. Extremely constrained links may also use an adaptive policy: send the small code normally and fall back to int8 or raw data when an anomaly detector is uncertain.

5.6 Privacy, Security and Resilience

Reducing or transforming sensor values may reduce direct exposure, but it does not guarantee privacy. A captured latent code is intentionally predictive of activity and can therefore disclose that activity. An attacker with the AE decoder or an auxiliary model may reconstruct properties of the original signal. Encryption in transit, endpoint authentication, least-privilege access and data retention controls remain necessary. Formal privacy would require a stated threat model and mechanisms such as differential privacy, secure aggregation or trusted execution, none of

which are evaluated here.

Lossy encoding can introduce resilience risks. Int8 calibration can fail after sensor replacement or distribution drift. A learned encoder may behave unpredictably on unseen activities, faults or adversarial inputs. A gateway should distinguish “one of six known activities” from “input is in distribution.” Confidence thresholds, range checks and a safe fallback are operational controls, but their false-alarm and missed-detection rates must be evaluated before a safety-related deployment.

5.7 Threats to Validity

5.7.1 Internal Validity

Training-only fitting, the official split and fixed seeds reduce leakage and accidental inconsistency. Nevertheless, the AE is evaluated with one principal seed and a finite hyperparameter search. Neural optimisation variance could alter small differences, especially around the PCA–AE crossover. Repeated runs with confidence intervals would strengthen the claim. The small positive int8 delta is not tested for statistical significance and is correctly interpreted as equivalent rather than superior.

5.7.2 Construct Validity

Payload bytes and Macro–F1 capture central objectives but not complete system cost. They omit headers, packet loss, radio airtime, encoder energy, memory peaks and tail latency. The UCI representation is engineered rather than raw; therefore the study evaluates post-feature compression. The reconstruction metrics are diagnostic and should not be interpreted as perceptual sensor fidelity.

5.7.3 External Validity

One dataset, six activities and a desktop gateway simulation limit generalisability. Other sensors may have different numeric ranges, redundancy or class boundaries. Cross-dataset HAR, streaming drift and physical gateways could shift the frontier. The results are best understood as evidence that the approach is feasible

and as a reproducible comparison method, not as a universal compression standard.

5.7.4 Conclusion Validity

The test partition contains many records but only a limited number of unseen subjects. Per-window observations from one subject are correlated, so naive bootstrap intervals over windows would be overconfident. Future uncertainty analysis should resample or cross-validate by subject. The current report avoids unsupported significance claims and emphasises large engineering differences, such as fourfold bytes at unchanged quality or matched-code gaps of several percentage points.

Chapter 6

Conclusion and Future Work

6.1 Conclusion

This project asked whether compressed IoT sensor data can support machine-learning inference at a nearby gateway. On the UCI HAR benchmark, the answer is yes, but the useful method depends on the permitted quality loss and endpoint capacity. The raw 561-float feature vector occupies 2,244 bytes and supports 95.443% Macro-F1. Record-wise zlib is exact but reduces the payload by only $1.14\times$. Float16 halves the message without an observed aggregate change. Feature-wise symmetric int8 sends 561 bytes and retains 95.502% Macro-F1, making it the strongest practical default.

Representation compression provides a different operating region. PCA and AE codes reduce the payload by up to $70.13\times$ but introduce task loss. PCA is stronger at 128 and 64 dimensions; the AE is stronger at 32 dimensions and below. AE-32 sends 128 bytes with 90.539% Macro-F1, and AE-8 sends 32 bytes with 86.773%. This crossover demonstrates the value of matched-byte comparison and the danger of assuming that a learned model always dominates a linear baseline.

The final recommendation is conditional. Use int8 when near-baseline activity quality is required. Use a compact AE code when the link is severely constrained, some quality loss is acceptable and the endpoint can afford a neural encoder. Keep model-artifact compression separate from message compression: dynamic int8 reduced the TorchScript classifier size, but its desktop latency did not improve. Across all choices, report the accounting boundary and evaluate downstream task quality rather than reconstruction or compression ratio alone.

6.2 Future Work

The first extension is hardware validation. The encoder, radio and gateway model should be deployed on a specified microcontroller, smartphone or Raspberry Pi, with energy, peak memory and end-to-end latency measured under controlled batch sizes and link conditions. Packet headers, retransmissions and encryption should be included so that feature-payload savings can be translated into actual airtime and battery impact.

The second extension is broader validity. Repeating the protocol on raw inertial windows, other HAR datasets and non-HAR sensor tasks would show whether int8 robustness and the PCA–AE crossover persist. Evaluation should use subject-level cross-validation or multiple official datasets, repeated neural seeds and confidence intervals that respect within-subject dependence.

The third extension is task-aware compression. A jointly trained encoder can combine reconstruction, classification and rate terms, and can potentially preserve more label information at 32 or 16 dimensions. Such a system should be compared with the simple baselines here at identical wire formats and should measure whether task specificity prevents reuse for other gateway analytics.

Finally, privacy and robustness require explicit experiments. Latent inversion, attribute inference, calibration drift, sensor faults and unknown activities should be treated as testable threat models. Adaptive fallback from a compact code to int8 or raw features may improve resilience, but its bandwidth and false alarm cost must be quantified. These extensions would turn the present controlled communication–quality study into a complete edge deployment evaluation.

Bibliography

- [1] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, “Edge computing: Vision and challenges,” *IEEE Internet of Things Journal*, vol. 3, no. 5, pp. 637–646, 2016.
- [2] M. Satyanarayanan, “The emergence of edge computing,” *Computer*, vol. 50, no. 1, pp. 30–39, 2017.
- [3] S. Deng, H. Zhao, W. Fang, J. Yin, S. Dustdar, and A. Y. Zomaya, “Edge intelligence: The confluence of edge computing and artificial intelligence,” *IEEE Internet of Things Journal*, vol. 7, no. 8, pp. 7457–7469, 2020.
- [4] D. Anguita, A. Ghio, L. Oneto, X. Parra, and J. L. Reyes-Ortiz, “A public domain dataset for human activity recognition using smartphones,” in *Proceedings of the 21st European Symposium on Artificial Neural Networks, Computational Intelligence and Machine Learning*, 2013, pp. 437–442.
- [5] J. L. Reyes-Ortiz, D. Anguita, A. Ghio, L. Oneto, and X. Parra, “Human activity recognition using smartphones,” 2012. [Online]. Available: <https://archive.ics.uci.edu/dataset/240/human+activity+recognition+using+smartphones>
- [6] Y. Kang, J. Hauswald, C. Gao, A. Rovinski, T. Mudge, J. Mars, and L. Tang, “Neurosurgeon: Collaborative intelligence between the cloud and mobile edge,” in *Proceedings of the 22nd International Conference on Architectural Support for Programming Languages and Operating Systems*, 2017, pp. 615–629.
- [7] S. Teerapittayanon, B. McDanel, and H. T. Kung, “Distributed deep neural networks over the cloud, the edge and end devices,” in *2017 IEEE 37th International Conference on Distributed Computing Systems*, 2017, pp. 328–339.
- [8] H. Choi and I. V. Bajic, “Deep feature compression for collaborative object detection,” *arXiv preprint arXiv:1802.03931*, 2018.

BIBLIOGRAPHY

- [9] J. Shao and J. Zhang, “BottleNet++: An end-to-end approach for feature compression in device-edge co-inference systems,” in *2020 IEEE International Conference on Communications Workshops*, 2020, pp. 1–6.
- [10] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, “Quantization and training of neural networks for efficient integer-arithmetic-only inference,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2018, pp. 2704–2713.
- [11] V. Sze, Y.-H. Chen, T.-J. Yang, and J. S. Emer, “Efficient processing of deep neural networks: A tutorial and survey,” *Proceedings of the IEEE*, vol. 105, no. 12, pp. 2295–2329, 2017.
- [12] I. T. Jolliffe and J. Cadima, “Principal component analysis: A review and recent developments,” *Philosophical Transactions of the Royal Society A*, vol. 374, no. 2065, p. 20150202, 2016.
- [13] G. E. Hinton and R. R. Salakhutdinov, “Reducing the dimensionality of data with neural networks,” *Science*, vol. 313, no. 5786, pp. 504–507, 2006.
- [14] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay, “Scikit-learn: Machine learning in python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [15] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Kopf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, “Pytorch: An imperative style, high-performance deep learning library,” in *Advances in Neural Information Processing Systems*, vol. 32, 2019.

Appendix A

Reproducibility Details

A.1 Repository Structure

The experiment source is located in the `code` directory. Dataset files are stored under `code/data/raw`; shared data, compression and metric utilities are under `code/src`; phase entry points are in the directory root; and each phase writes to a dedicated directory under `code/outputs`. Report figures are referenced directly from these generated outputs, which reduces manual transcription.

Table A.1: Experiment entry points and principal outputs.

Command	Purpose and output
<code>run_phase1.py</code>	Load/validate UCI HAR, fit raw baselines, measure zlib and write <code>outputs/phase1</code> .
<code>run_phase2_quantization.py</code>	Evaluate float16 and feature-wise int8; write quantisation metrics and plot.
<code>run_phase3_pca.py</code>	Fit training-only PCA models at five dimensions; write metrics and artifacts.
<code>run_phase4_autoencoder.py</code>	Train five AEs with validation and early stopping; export latent results and curves.
<code>run_phase5_analysis.py</code>	Merge families, calculate the frontier and generate recommendation plots.
<code>run_phase6_deployment.py</code>	Export float and dynamic-int8 TorchScript classifiers and validate the end-to-end route.

A.2 Environment and Dependencies

The implementation uses Python, NumPy, pandas, scikit-learn [14], Matplotlib, joblib and PyTorch [15]. Exact direct dependencies are declared in

`requirements.txt`; `requirements-lock.txt` captures the resolved environment used for the completed run. A clean environment should install the lock file where the Python version and platform are compatible. The project records library versions in its reproducibility output because serialization compatibility can vary across scikit-learn and PyTorch releases.

The original interim report proposed TensorFlow Lite. The final executable proxy uses TorchScript because the implemented AE and deployment pipeline are PyTorch based and the available environment supports reproducible export. This is a documented implementation change, not evidence that TorchScript is universally preferable. A target-specific study can convert or retrain the selected model for TensorFlow Lite and compare runtime kernels directly.

A.3 Determinism and Data Integrity

The dataset loader checks that training and test matrices have 561 columns and that label counts and subject files align. The official split is preserved. Seed 5105 is used for stochastic estimators, validation splitting and PyTorch initialisation. Deterministic seeds improve traceability but do not eliminate all platform-level variation in parallel numerical libraries. Repeating AE training and reporting mean and standard deviation is recommended for a publication-level extension.

Every fitted transform is trained without test records. Int8 scales are derived from the training maximum absolute value of each feature. PCA and standardisation are fitted on training data. The AE validation subset is drawn only from the official training set. Test labels are read for final metrics but never used in encoder fitting, early stopping or recommendation threshold construction.

A.4 Artifact Checks

Phase outputs include CSV/JSON metrics, confusion matrices, serialized models and plots. Reproduction should verify that metric files parse, image files are non-empty and the unified table contains one row for every expected method. The TorchScript float export must reproduce scikit-learn predictions before dynamic quantisation is interpreted. The float16 round trip should remain finite; the int8

APPENDIX A. REPRODUCIBILITY DETAILS

decoder should apply the same scale version as the endpoint.

The key expected values, allowing only minor last-decimal variation, are:

- raw Macro-F1 approximately 0.95443 with 2,244 bytes;
- int8 Macro-F1 approximately 0.95502 with 561 bytes;
- PCA-32 Macro-F1 approximately 0.88931 with 128 bytes;
- AE-32 Macro-F1 approximately 0.90539 with 128 bytes;
- end-to-end feature-int8 plus dynamic-model-int8 Macro-F1 approximately 0.95035.

Large deviations should be investigated before updating the report. Common causes include fitting a transform on combined data, changing a random seed, using a different official split, or loading an incompatible serialized artifact.

A.5 Payload Formulae

The report’s fixed-width payload calculations are reproducible without inspecting file sizes:

$$B_{float32} = 561 \times 4 = 2244, \tag{A.1}$$

$$B_{float16} = 561 \times 2 = 1122, \tag{A.2}$$

$$B_{int8} = 561 \times 1 = 561, \tag{A.3}$$

$$B_{latent}(k) = k \times 4. \tag{A.4}$$

zlib is the only variable-length family and is therefore measured empirically per test record. Container files, pickles and model artifacts are not substituted for wire payloads. Gateway artifacts are reported in their own deployment table.